

Exercise 1 (Interactive Exploration of Parameters in ISOMAP)

The focus of this exercise is to learn about ISOMAP embeddings and how they vary if we modify the neighborhood size and the level of noise in the data. We will be using the swiss roll dataset. To come along with the exercise we will need **IPyWidgets**, that helps us to interactively set the parameters of a visualization in a Jupyter notebook.

```
In [111... import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns
```

a) Generating the swiss roll data

```
In [112... from sklearn.datasets import make_swiss_roll

# Generating the data:
df,t=make_swiss_roll(n_samples=1000, noise=0.42, random_state=42)
```

- df = the 3D coordinates of points on the rolled surface (shape: 1000x3)
- t = the position along the strip before it was rolled, it indicates how far along the roll the point is (shape: 1000x1)

```
In [113... # Plotting:

sns.set(style="whitegrid")

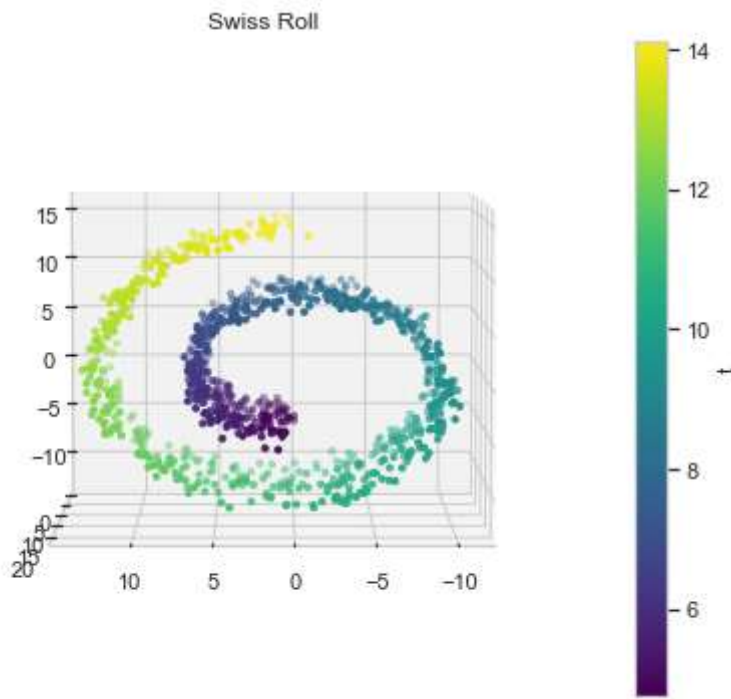
mpl.rcParams['axes.grid'] = False #there was a warning popping out

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection='3d')

scatter = ax.scatter(df[:, 0], df[:, 1],df[:, 2],
                    c=t, s=9, cmap='viridis')

ax.view_init(elev=5, azim=90)

plt.colorbar(scatter, label="t")
plt.title("Swiss Roll")
plt.show()
```



b) Unrolling the roll

In [114...

```
from sklearn.manifold import Isomap

# Unrolled points:
iso=Isomap(n_neighbors=7,n_components=2)
df_iso=iso.fit_transform(df) #shape: (1000,2)
D = iso.dist_matrix_ # geodesic distance matrix

# The points that are more to the left:
index_min=np.argmin(df_iso[:,0])
geo_dist = D[index_min] #
left_point_2d=df_iso[index_min,:]
left_point_3d=df[index_min,:]

# Both plots, colored by geodesic distance:

fig = plt.figure(figsize=(14, 5))

# 1) 3D SWISS ROLL

ax1 = fig.add_subplot(1, 2, 1, projection='3d')

scatter = ax1.scatter(df[:, 0], df[:, 1], df[:, 2],c=geo_dist, s=8, cmap='viridis')
ax1.scatter(left_point_3d[0], left_point_3d[1], left_point_3d[2], c='red', s=30)

ax1.view_init(elev=5, azim=90)
ax1.set_title("Swiss Roll")

fig.colorbar(scatter, ax=ax1)

# 2) 2D ISOMAP

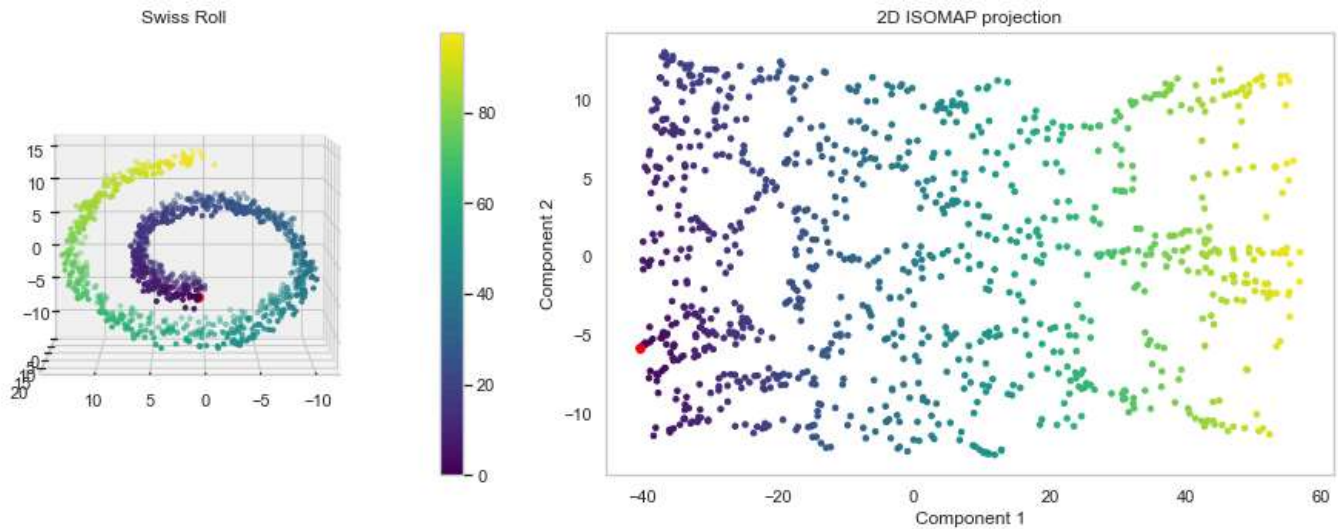
ax2 = fig.add_subplot(1, 2, 2)

ax2.scatter(df_iso[:, 0], df_iso[:, 1], c=geo_dist, s=12, cmap='viridis')
ax2.scatter(left_point_2d[0], left_point_2d[1], c='red', s=30)

ax2.set_title("2D ISOMAP projection")
```

```
ax2.set_xlabel('Component 1')
ax2.set_ylabel('Component 2')

plt.tight_layout()
plt.show()
```



To choose the number of neighbors k we took into account that:

- If k is too small: the graph breaks into pieces, it become disconnected
- If k is too large: shortcuts appear, manifold gets flattened incorrectly

c) Interactive controls

In this part we will initialize a slider for the standard deviation in the swiss roll data generator, the number of neighbors for building the neighborhood graph in the ISOMAP algorithm, and an index over all data points, to set the reference for color coding distances.

```
In [115... from ipywidgets import interact, interactive, fixed, interact_manual
import ipywidgets as widgets
from IPython.display import display
```

```
In [116... std_slider = widgets.FloatSlider(value=0.42, min=0.0, max=1.0, step=0.01, description="std")
nn_slider = widgets.IntSlider(value=8, min=1, max=25, step=1, description="k")
index_slider = widgets.IntSlider(value=0, min=0, max=999, step=1, description="index")
```

d) Implementing the sliders

Because we want to be able to vary the input that generates the data and the plots, we need to define a function that generates all the outputs we obtained in the last parts:

```
In [117... def isomapping_swissroll (std,n_neighbors,index):

    # Data:
    df,t=make_swiss_roll(n_samples=1000, noise=std, random_state=42)

    # Unrolling points:
    iso = Isomap(n_neighbors=n_neighbors, n_components=2)
    df_iso = iso.fit_transform(df)
    D = iso.dist_matrix_

    # Reference point:
```

```

sorted_indices = np.argsort(df_iso[:, 0])
sorted_distances = D[sorted_indices]
sorted_data= df[sorted_indices]
sorted_data_iso= df_iso[sorted_indices]
geo_dist=sorted_distances[index]

# Plots:
fig = plt.figure(figsize=(14, 5))

# 1) 3D SWISS ROLL

ax1 = fig.add_subplot(1, 2, 1, projection='3d')

scatter = ax1.scatter(df[:, 0], df[:, 1], df[:, 2], c=geo_dist, s=8, cmap='viridis')
ax1.scatter(sorted_data[index,0], sorted_data[index,1], sorted_data[index,2], c='red', s=50)

ax1.view_init(elev=5, azim=90)
ax1.set_title("Swiss Roll")

fig.colorbar(scatter, ax=ax1, label="t")

# 2) 2D ISOMAP

ax2 = fig.add_subplot(1, 2, 2)

ax2.scatter(df_iso[:, 0], df_iso[:, 1], c=geo_dist, s=12, cmap='viridis')
ax2.scatter(sorted_data_iso[index,0], sorted_data_iso[index,1], c='red', s=50)

ax2.set_title("2D ISOMAP projection")
ax2.set_xlabel('Component 1')
ax2.set_ylabel('Component 2')

plt.tight_layout()
plt.show()

# CONECTINC THE SLIDERS TO THE FUNCTIONS:

widgets.interact(isomapping_swissroll,std=std_slider,n_neighbors=nn_slider,index=index_slider)

interactive(children=(FloatSlider(value=0.42, description='std', max=1.0, step=0.01), IntSlider
(value=8, descr...
<function __main__.isomapping_swissroll(std, n_neighbors, index)>

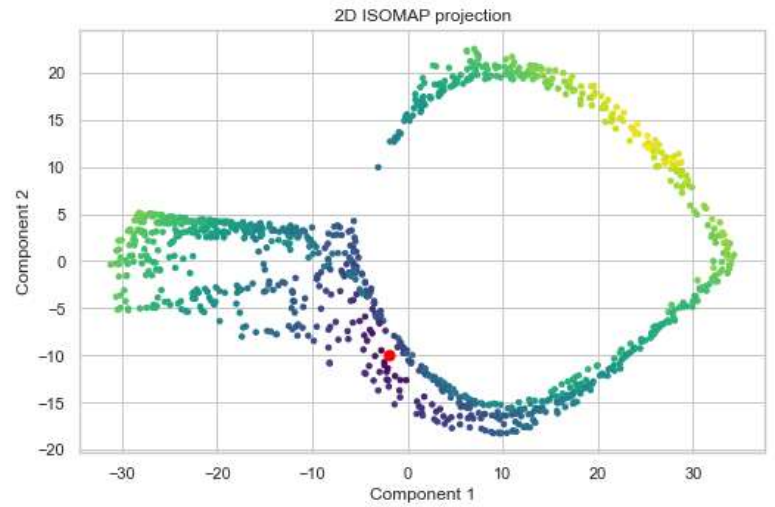
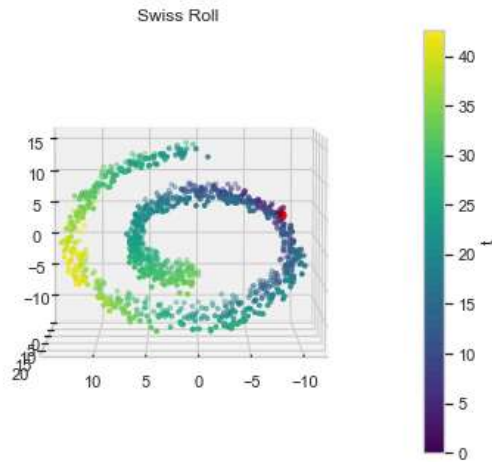
```

Out[117]:

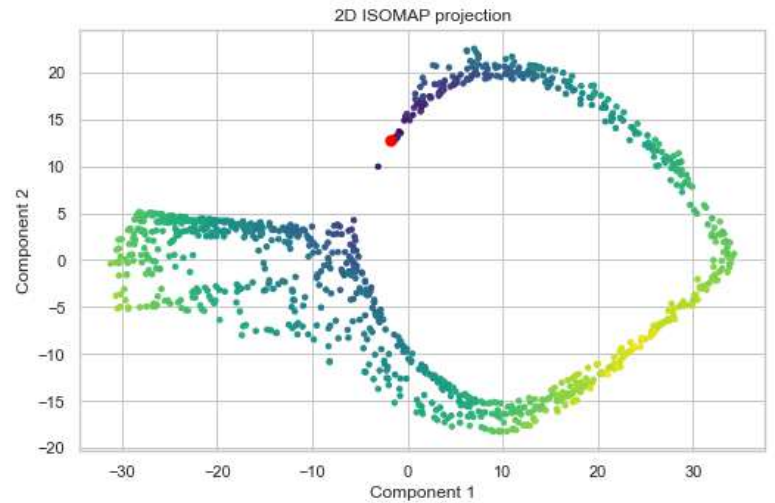
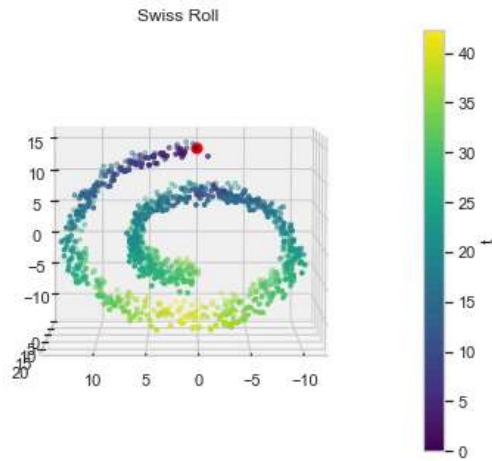
e) Fixed k, different std

std=0.45, the first value of std that fails to reconstruct the true underlying manifold with k=8.

Index: 466



Index: 467



The problematic region is found in the bifurcation of the 2D projection located approximately in (-6,5). The points form the two different branches belong to two very different regions of the original manifold. Because the number of neighbors may be too high, the algorithm incorrectly connects distant parts of the manifold, leading to an inaccurate reconstruction of the geodesic distances.

Trying to solve the problem, we choose $k=7$. With less neighbors, the algorithm works better in the problematic regions because it truly takes the closest points to reconstruct the structure and is not forced to take more points than needed, which might not be respecting the original structure.

